PSR-12

PSR-12 is built on PSR-1; in fact, the first rule of PSR-12 is that all code must follow PSR-1. It also adds to the standards for storing/transmitting source files by specifying the line termination character (Unix line-feed) and the removal of the closing tag sequence `?>`. Perhaps controversially, PSR-12 also insists on four-space indentation (indent style is often the cause of many robust conversations among programmers).

PSR-12 contains a lot of fine detail on how to format things like class names, function names, and constants; it also enforces the layout of control structures, loops, spacing for operators and operands, and spacing for function parameter lists. There's a lot of detail that is unnecessary to reproduce here, but you are strongly encouraged to read the PSR documents; they will be referred to throughout this book (which will have all code examples conforming to the PSR-12 standard except where it is necessary to break those standards as part of an example).

Enforcing Standards

The PSR-12 standard contains a lot of information, and even seasoned veterans that have been using it for years will occasionally slip up and miss something (ahem…), so it's a good idea to use a scanning tool to make sure that standards are maintained on every change. For this we have PHP CodeSniffer (`phpcs`), which is fairly ubiquitous and the code scanning tool of choice for many high-profile projects. **Appendix A** contains everything you need to install and run both CodeSniffer and its related `phpcbf` fixer utility.

Input (and Output)

Any program you create will naturally need to access, transform, and output data in some form. Numbers, text, even images, and other more exotic forms of information are the lifeblood of software. PHP has several mechanisms available to make this task as painless as possible. The acronym *I/O* ("input/ouput") is quite common and will be used here. This section will discuss things like bytes and byte streams. If you're not certain what these terms mean, I recommend a quick peek at Chapter 2, "Data, Types, and Variables," which introduces the basic principles of data representation. Before we get started with PHP's I/O mechanisms, we need to look at the differences in *how* PHP executes, because this impacts what is available and what needs to happen.